BUSCA INFORMADA

Resolução de problemas
por meio de buscas

Prof. Tacla UTFPR/Curitiba

Referência

- Capítulo 4 - Rusell & Norvig

Roteiro

Busca de melhor escolha (Best-first)

- GULOSA pela Melhor Escolha (Greedy best-first)
- A* (minimizar custo total *estimado* da solução) - Heurísticas admissíveis

BUSCA INFORMADA

- Busca informada utiliza conhecimento específico do problema para guiar a busca.

- Conhecimento específico é adicionado à definição (formulação) do problema.
  - Estado inicial,
  - modelo de transição (função sucessora),
  - custo de ação,
  - estado objetivo
  - *função heurística*

- Podem encontrar soluções de forma mais eficiente do que as buscas cegas
  - ao custo de serem mais específicos

Resolução de problemas por meio de busca

BUSCA DE MELHOR ESCOLHA (BEST-FIRST)

Busca Melhor Escolha (BEST-FIRST)

Melhor escolha é uma abordagem geral de busca informada.

Especializada em: Gulosa e A*

Utiliza uma **função de avaliação** **f(n)** para cada nó para estimar o quanto *n* é desejável para fazer parte da solução.

f(n) o nó que apresentar **menor f(n) é expandido primeiro.**

Implementação é idêntica ao da busca de custo uniforme substituindo-se g(n) por f(n)

Busca Melhor Escolha: f(n)

$$f(n) = g(n) + h(n)$$

g(n) = Custo do caminho do estado inicial até o nó **n**

h(n) = Custo estimado de **n** ao **estado objetivo** pelo caminho de menor custo.

Resolução de problemas por meio de buscas

BUSCA GULOSA (GREEDY-FIRST)

Busca Gulosa

- A cada passo tenta chegar mais perto do estado objetivo sem se preocupar com os passos dados.

- Utiliza somente a componente heurística da função f(n)

$$f(n) = \cancel{g(n)} + h(n)$$

- Logo, **f(n) = h(n)**

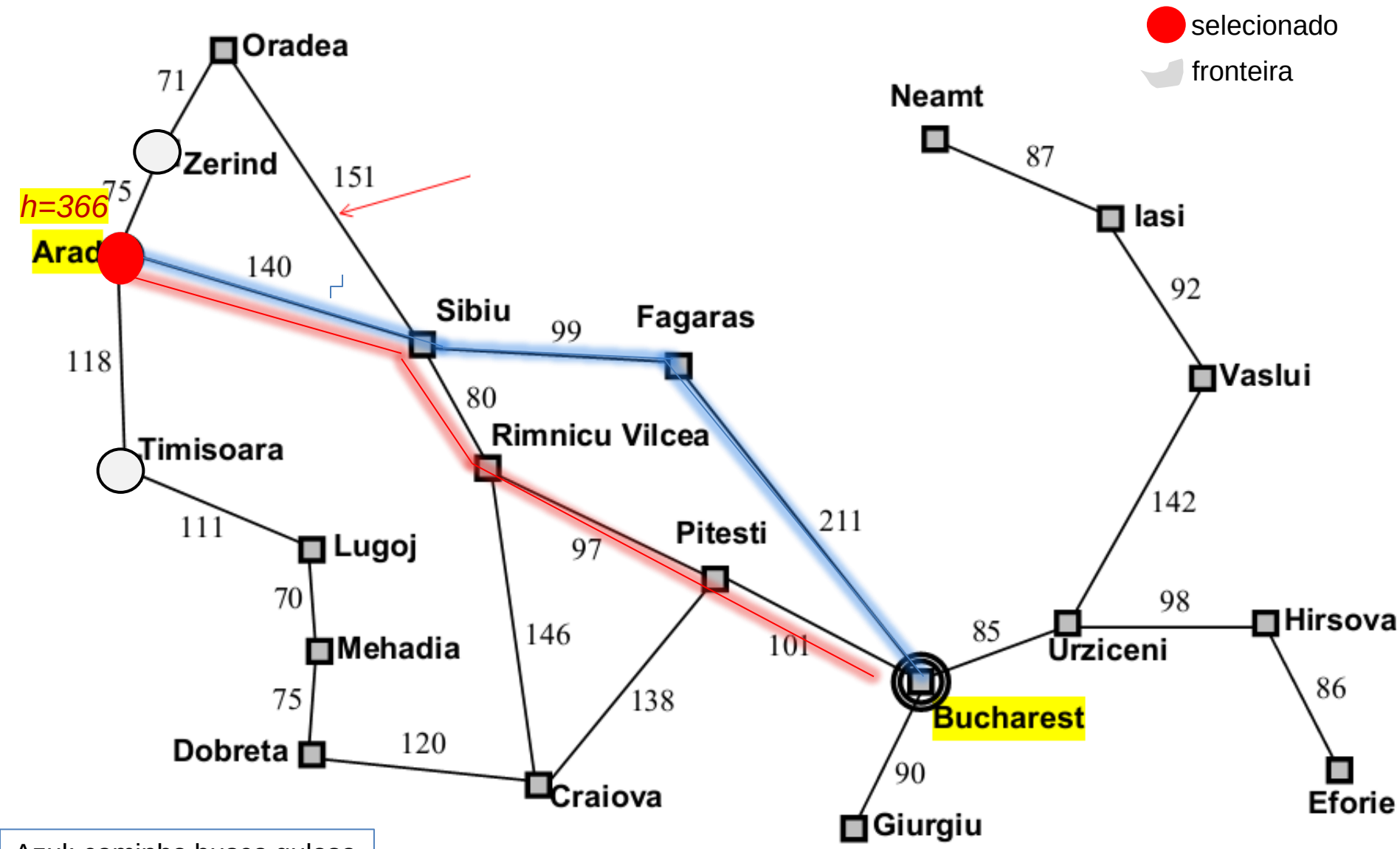
Exemplo de Busca Gulosa pela Melhor Escolha

h(n) = distâncias estimadas em linha reta até Bucareste

Arad	366	**Mehadia**	241
Bucharest	0	**Neamt**	234
Craiova	160	**Oradea**	380
Drobeta	242	**Pitesti**	100
Eforie	161	**Rimnicu Vilcea**	193
Fagaras	176	**Sibiu**	253
Giurgiu	77	**Timisoara**	329
Hirsova	151	**Urziceni**	80
Iasi	226	**Vaslui**	199
Lugoj	244	**Zerind**	374

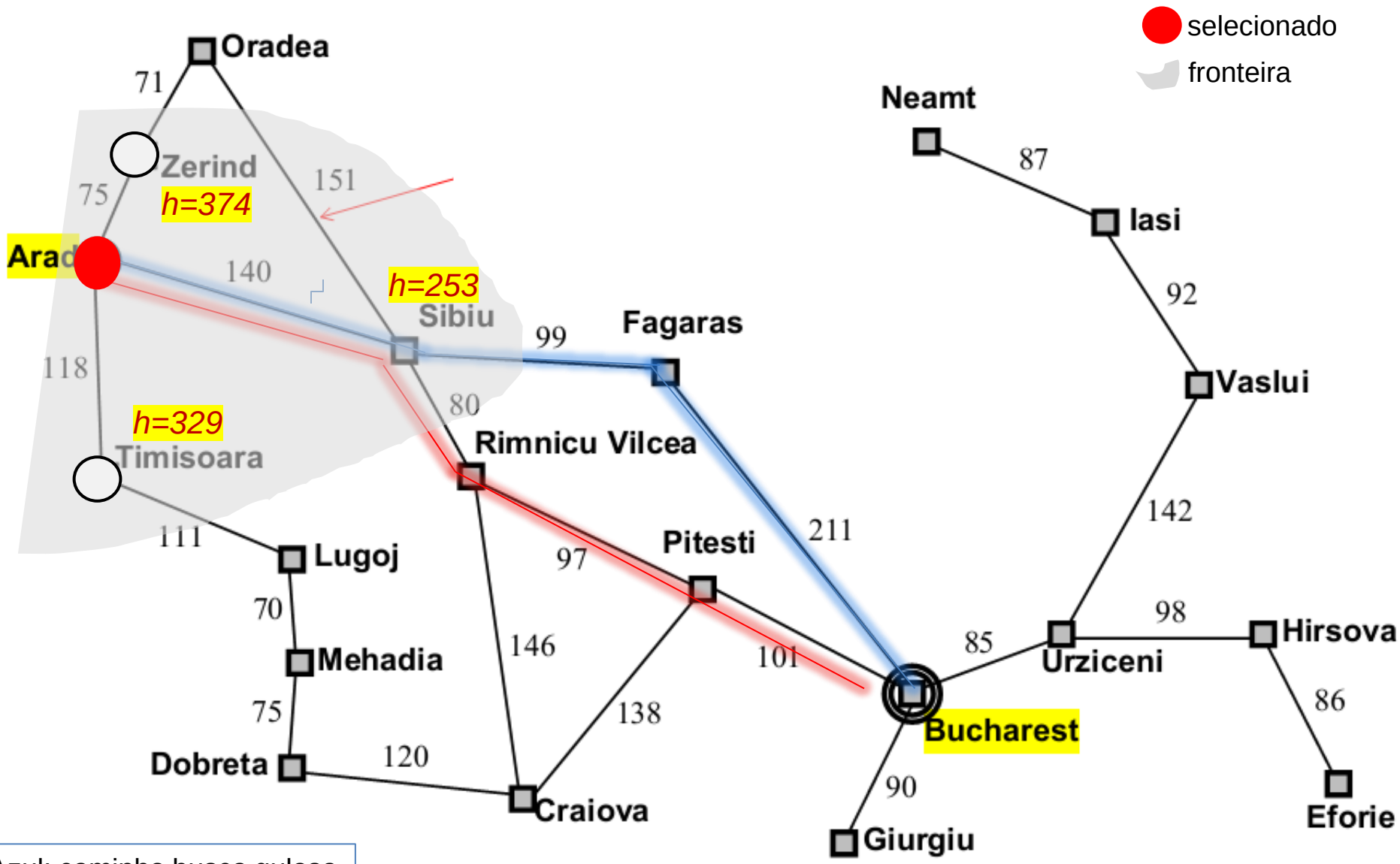
Fig. 3.22 Russel & Norvig

Exemplo de Busca Gulosa pela Melhor Escolha



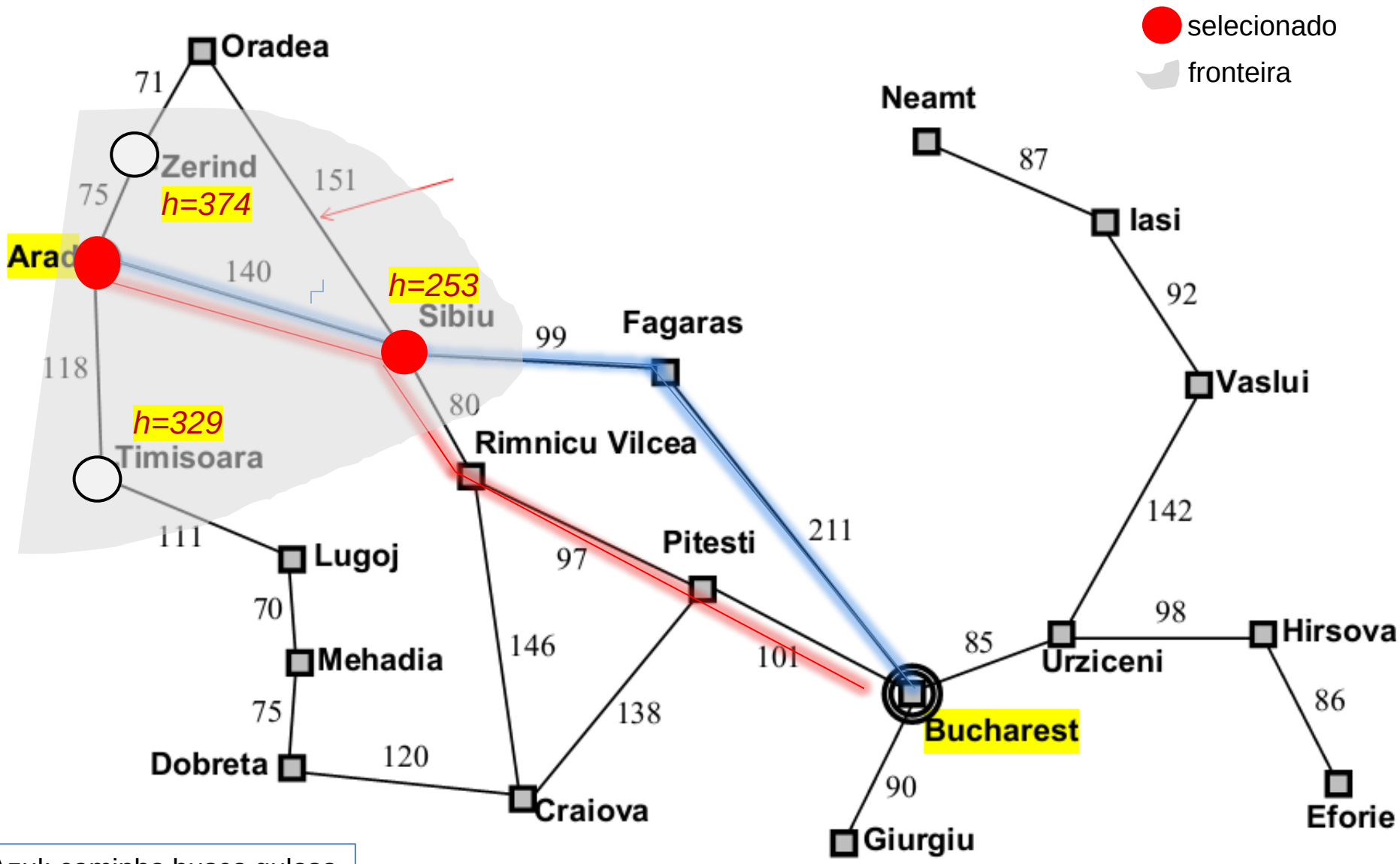
Azul: caminho busca gulosa
Vermelho: caminho ótimo

Exemplo de Busca Gulosa pela Melhor Escolha



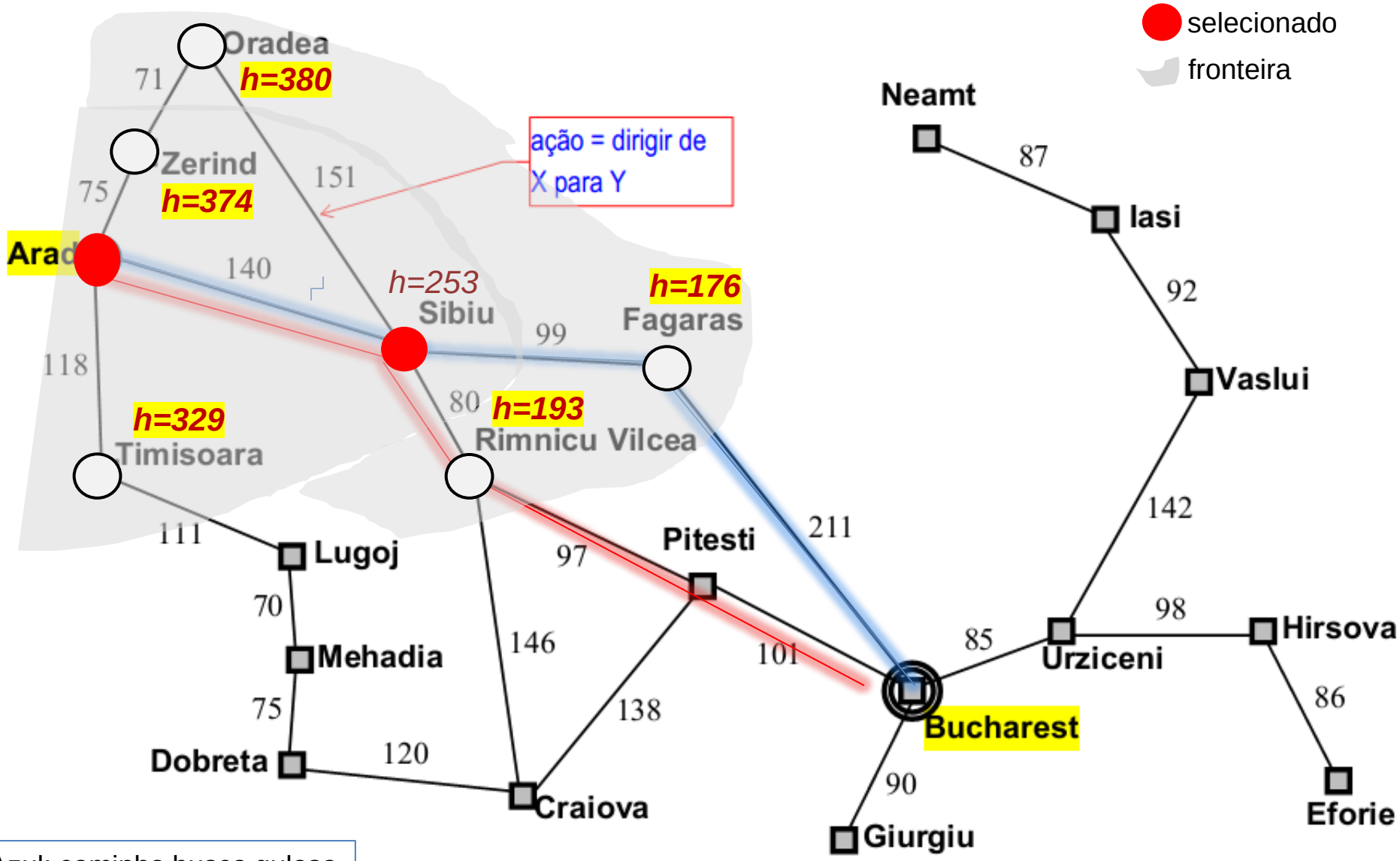
Azul: caminho busca gulosa
Vermelho: caminho ótimo

Exemplo de Busca Gulosa pela Melhor Escolha



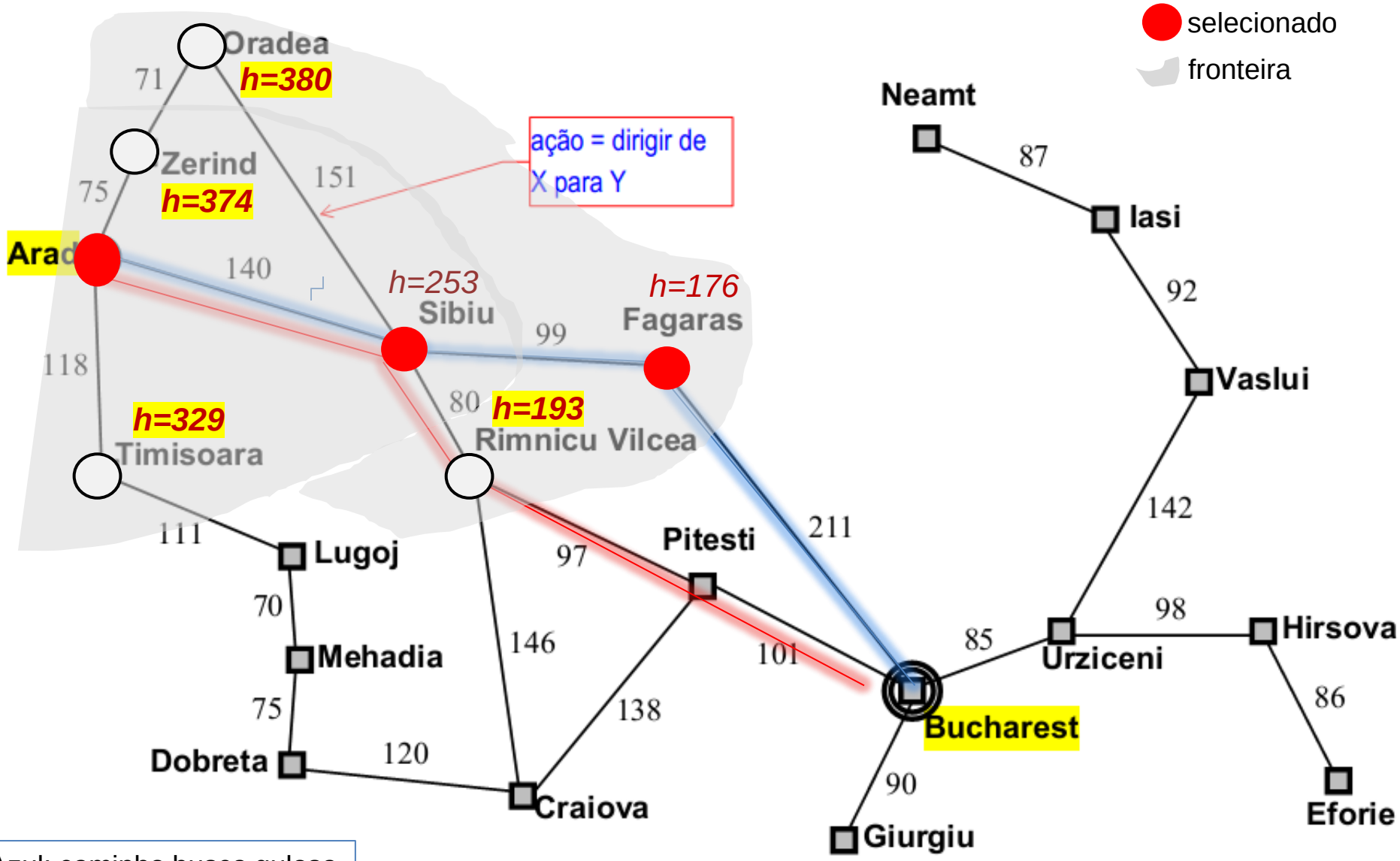
Azul: caminho busca gulosa
Vermelho: caminho ótimo

Exemplo de Busca Gulosa pela Melhor Escolha



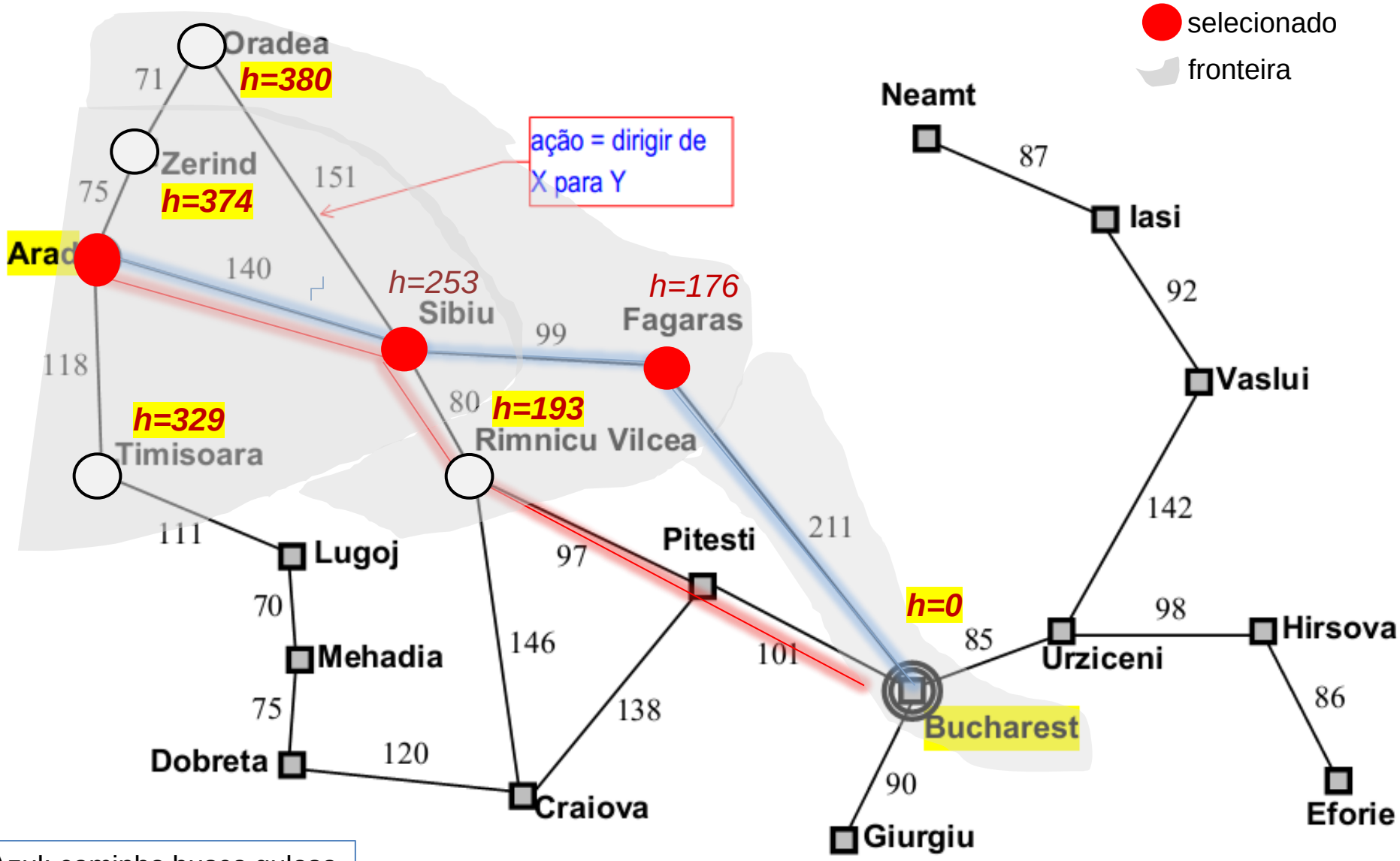
Azul: caminho busca gulosa
Vermelho: caminho ótimo

Exemplo de Busca Gulosa pela Melhor Escolha



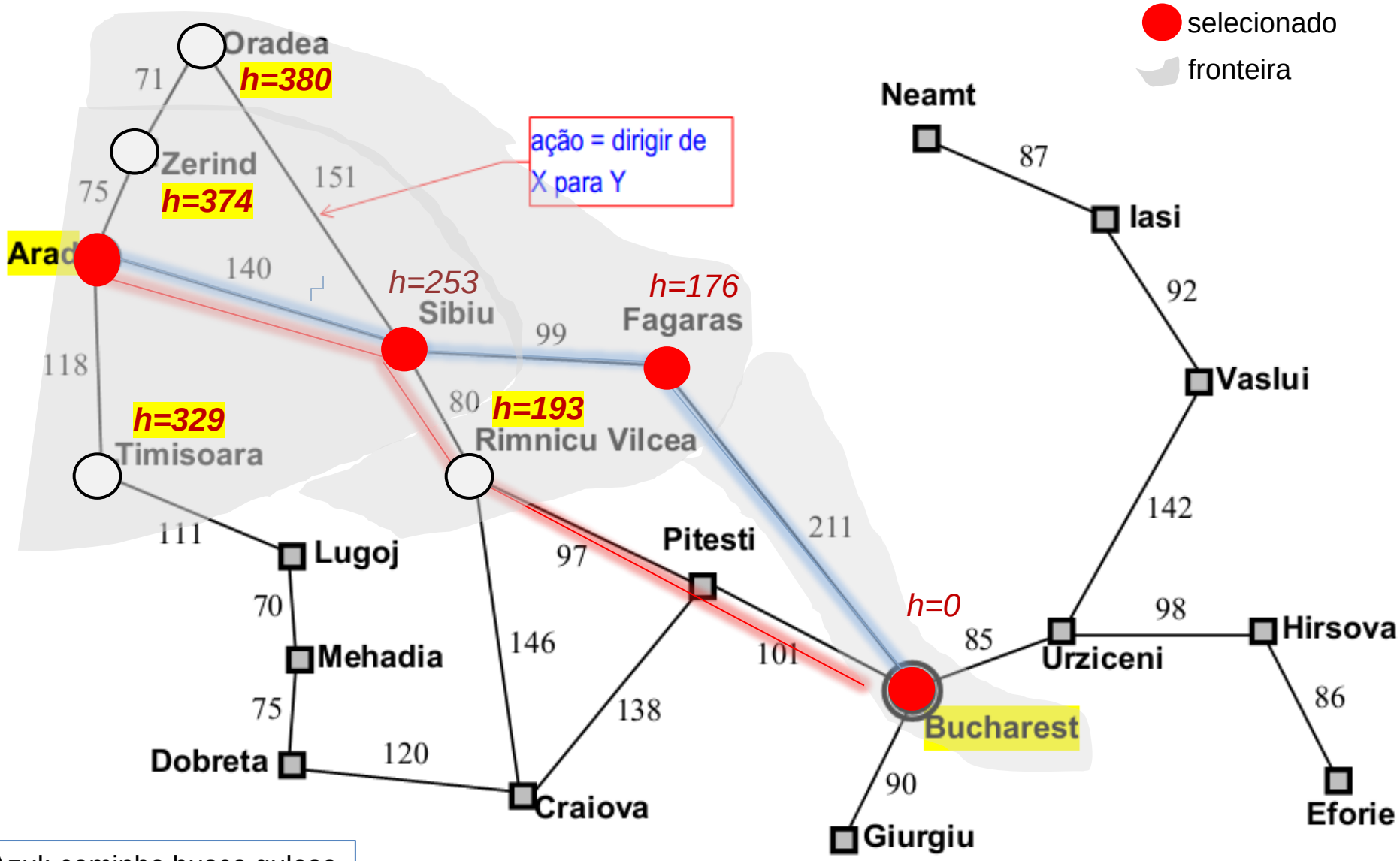
Azul: caminho busca gulosa
Vermelho: caminho ótimo

Exemplo de Busca Gulosa pela Melhor Escolha



Azul: caminho busca gulosa
Vermelho: caminho ótimo

Exemplo de Busca Gulosa pela Melhor Escolha



Azul: caminho busca gulosa
Vermelho: caminho ótimo

Avaliação da Busca Gulosa

Espacial	**$O(b^m)$**
Tempo	**$O(b^m)$**
Completo	**Sim**, para busca em grafo, se o espaço de estados for finito
Ótimo	**Não**

$O(b^m)$ no pior dos casos, o caminho mais promissor é aquele com custos heurísticos pequenos

Resolução de problemas por meio de buscas

A*

Busca A*

- **Ideia**: podar caminhos que são caros

- Função de avaliação $\mathbf{f(n) = g(n) + h(n)}$

  g(n) = custo para chegar ao nó *n*

  h(n) = custo estimado para ir de *n* até o objetivo

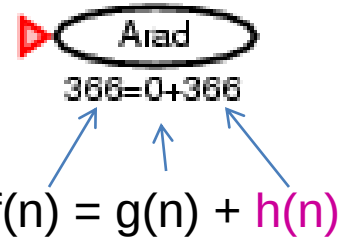
  f(n) = custo estimado total do caminho para chegar do estado inicial ao objetivo passando por *n*

Busca A*

Comparação do f(n) das estratégias de busca

```
            A*: f(n) = g(n) + h(n)
Custo Uniforme: f(n) = g(n)
       Gulosa: f(n) =        h(n)
```

Exemplo de Busca A*

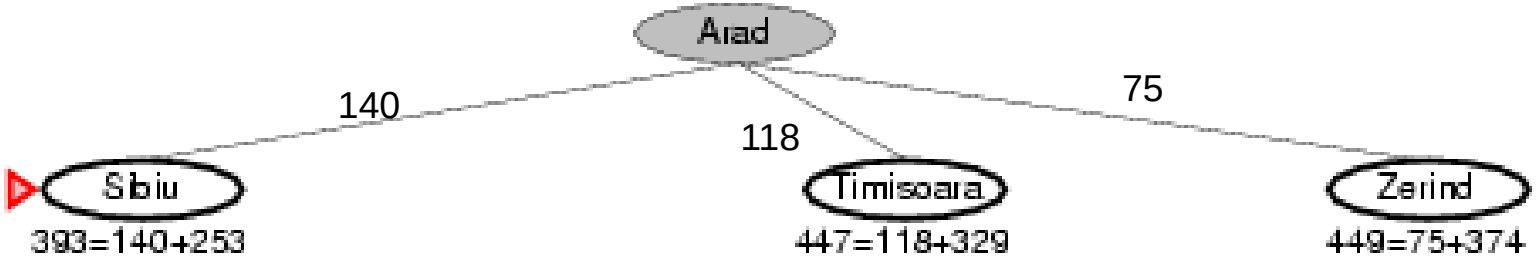


h(n) distância em linha reta de n até Bucareste

Fronteira lista ordenada por f(n)

Arad (366)

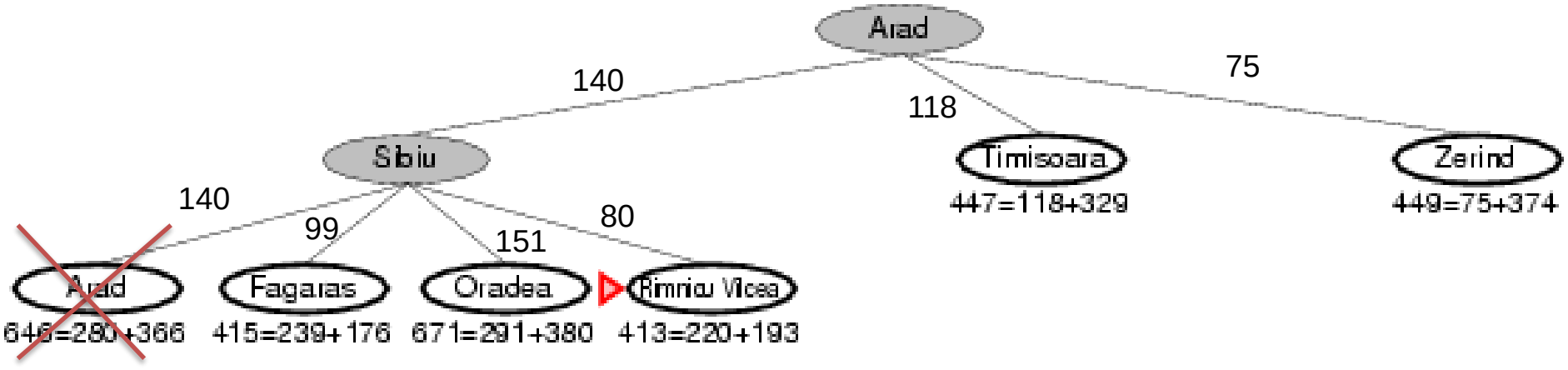
Exemplo de Busca A*



Fronteira lista ordenada por f(n)

Sibiu (393) < Timisoara (447) < Zerind (449)

Exemplo de Busca A*

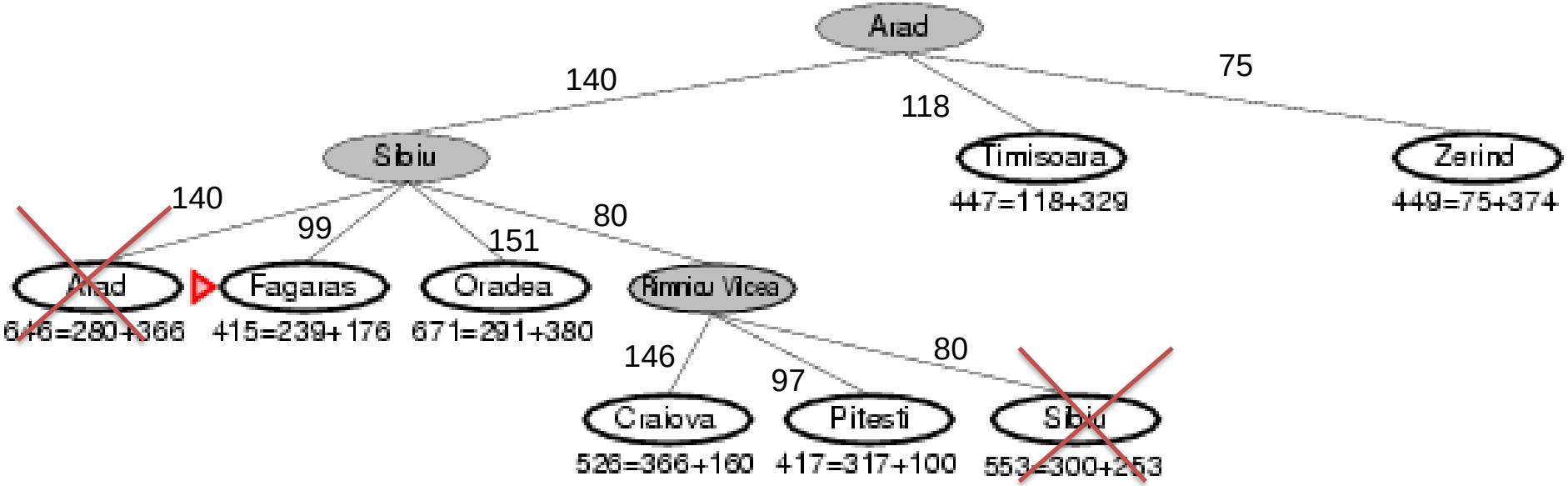


Estado já explorado; não está na fronteira

Fronteira lista ordenada por f(n)

Rimnicu (413) < Fagaras (415) < Timisoara (447) < Zerind (449) < Oradea (671)

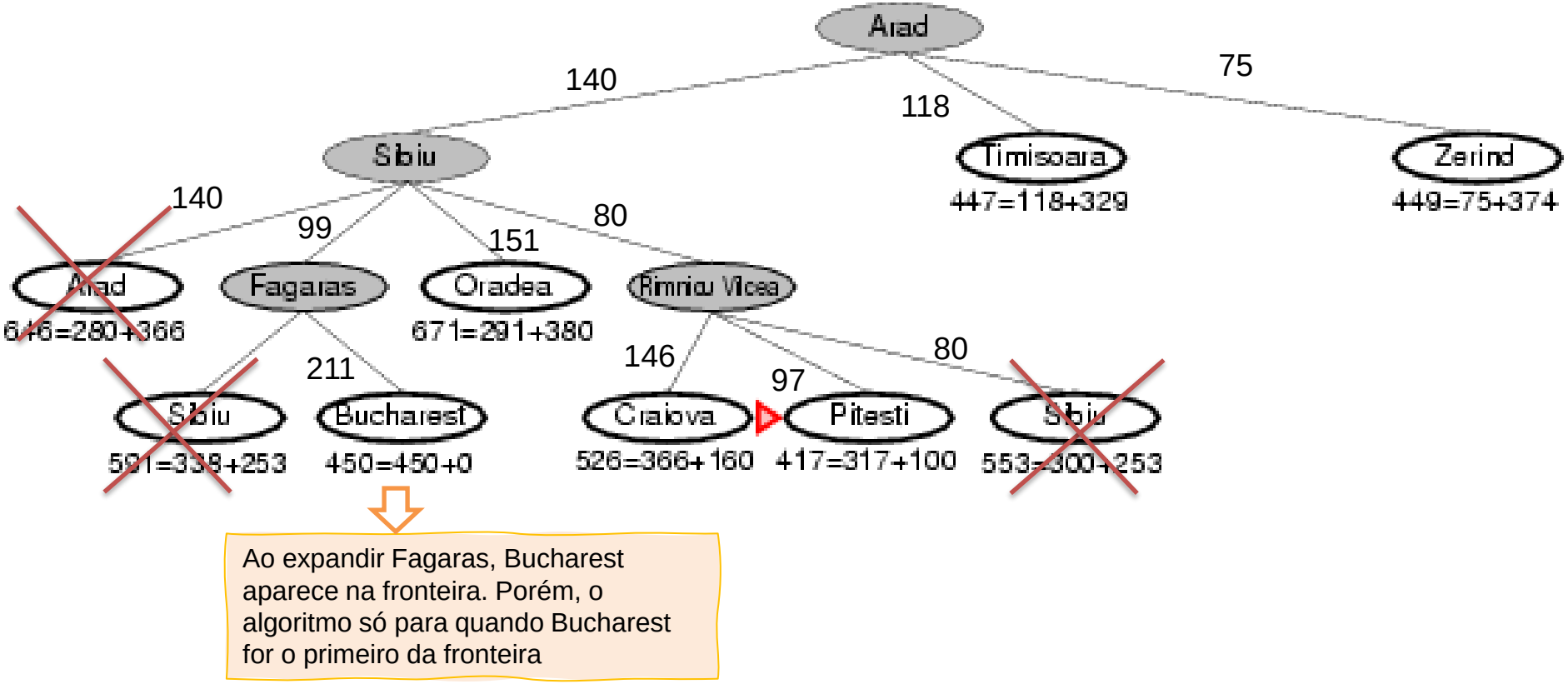
Exemplo de Busca A*



Fronteira lista ordenada por f(n)

Fagaras (415) < Pitesti (417) < Timisoara (447) < Zerind (449) < Craiova (526) < Oradea (671)

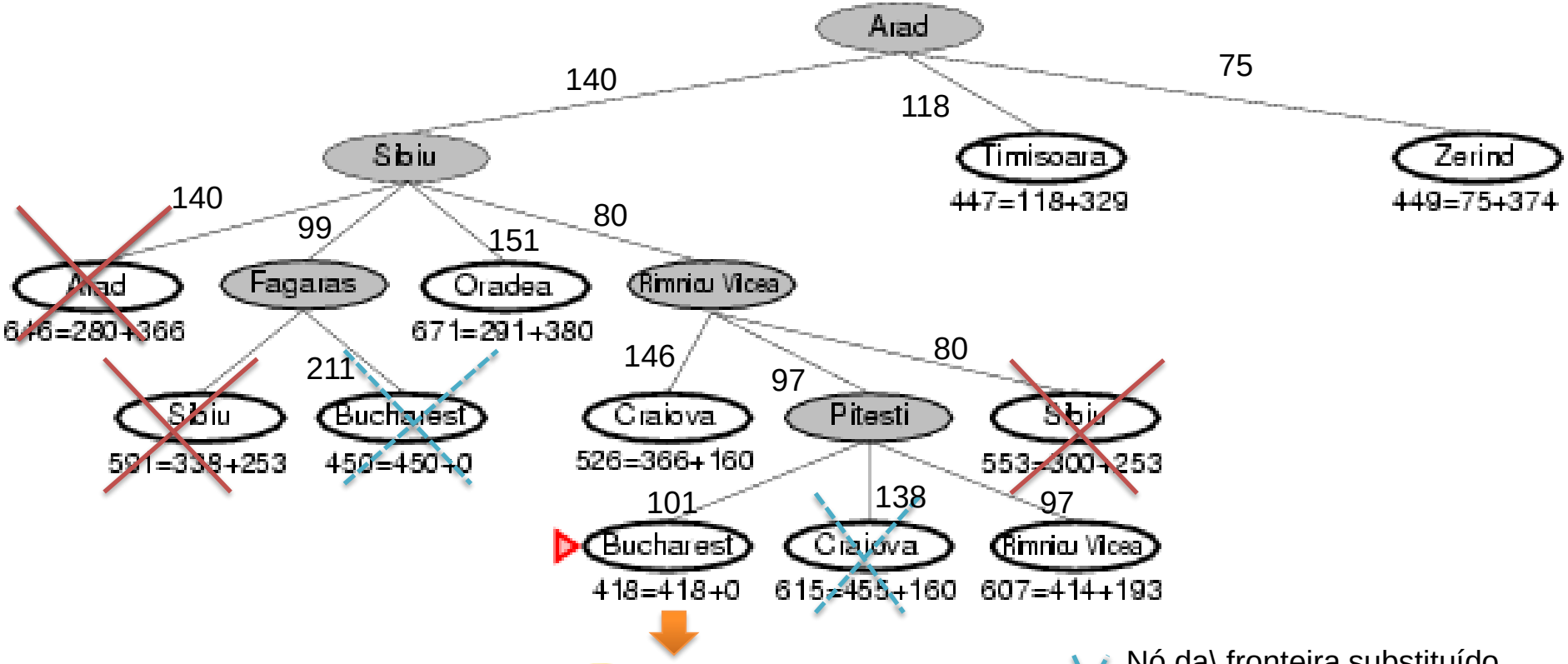
Exemplo de Busca A*



Fronteira lista ordenada por f(n)

Pitesti (417) < Timisoara (447) < Zerind (449) < **Bucharest (450)** < Craiova (526) < Oradea (671)

Exemplo de Busca A*



Fronteiral lista ordenada por f(n)

Bucharest (418) < Timisoara (447) < Zerind (449) < Craiova (526) < Rimnicu (607) < Oradea (671)

OTIMALIDADE DE A*

- A otimalidade de A* depende da componente h(n)

- Condições para otimalidade:
  - **h(n)** deve ser uma heurística **admissível**:
    - nunca **superestimar** o custo real para alcançar o estado objetivo
      Para busca em espaços de estados que são árvores basta ser admissível

  - **h(n)** deve ser **consistente**
    - **não pode decrescer**
    - *Para busca em espaços de estados que são GRAFOS deve ser admissível e consistente*

Heurística Admissível

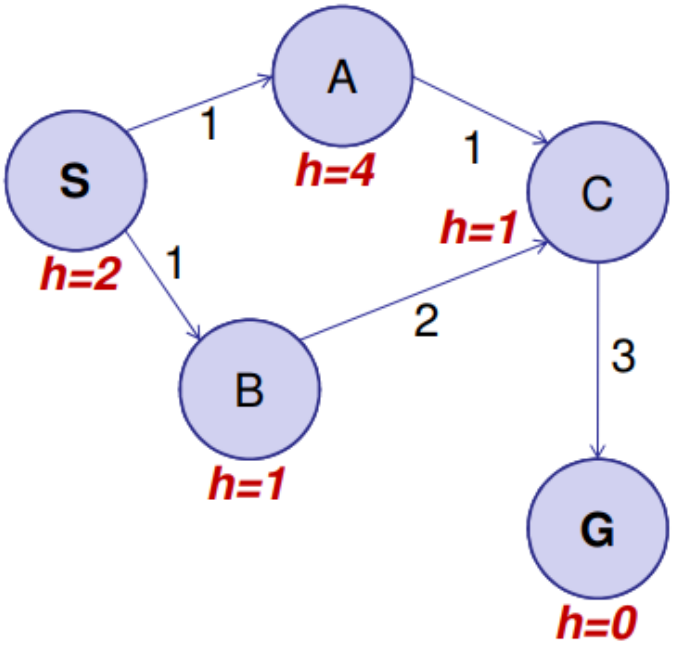
Uma heurística *h(n)* é admissível se todo nó *n*:

$$h(n) \leq h^*(n),$$

onde $h^*(n)$ é o custo "real" (modelado) para se alcançar o estado-objetivo a partir de n

DEVE SER **OTIMISTA**!

Heurística Admissível



Heurística admissível?

n	h(n)	h*(n)	h(n) ≤ h*(n)
S	2	5	V
A	4	4	V
B	1	5	V
C	1	3	V
G	0	0	V

OTIMALIDADE DE A*

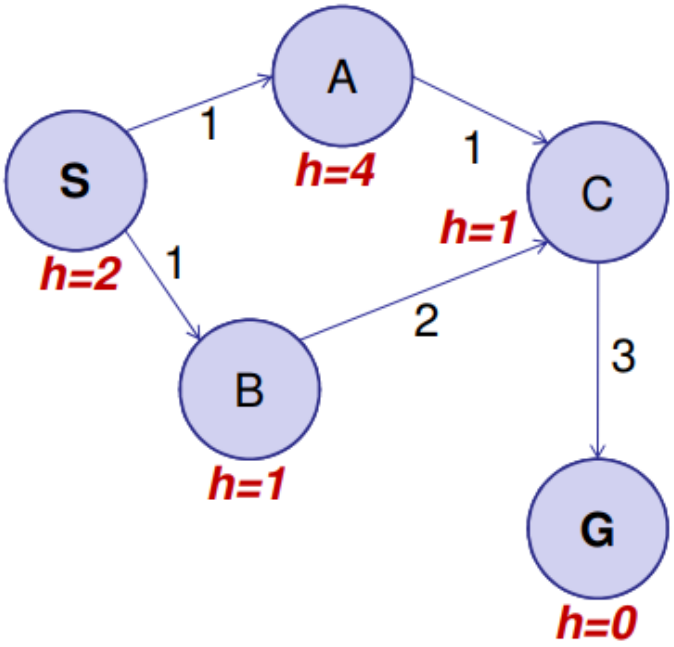
- A otimalidade de A* depende da componente h(n)

- Condições para otimalidade:
  - **h(n)** deve ser uma heurística **admissível**

  - **h(n)** deve ser **consistente**
    - **respeitar** o princípio da desigualdade triangular para garantir que f(n) não decresça em qualquer um dos caminhos
    - *Para busca em espaço de estados que sejam grafos tem que ser admissível e consistente*

$$h(n) \leq c(n,a,n') + h(n')$$

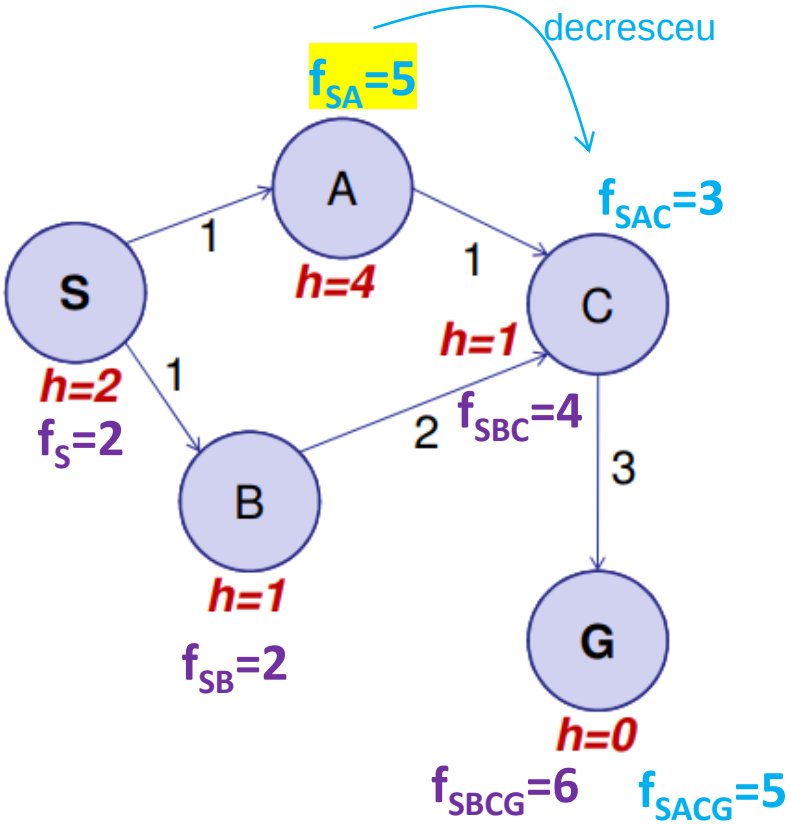
Heurística Consistente



Heurística consistente?

n	n’	h(n)	≤	c(n,a,n’)+h(n’)
S	A	2	V	5
S	B	2	V	2
A	C	4	F	2
B	C	1	V	3
C	G	1	V	3

Heurística Consistente



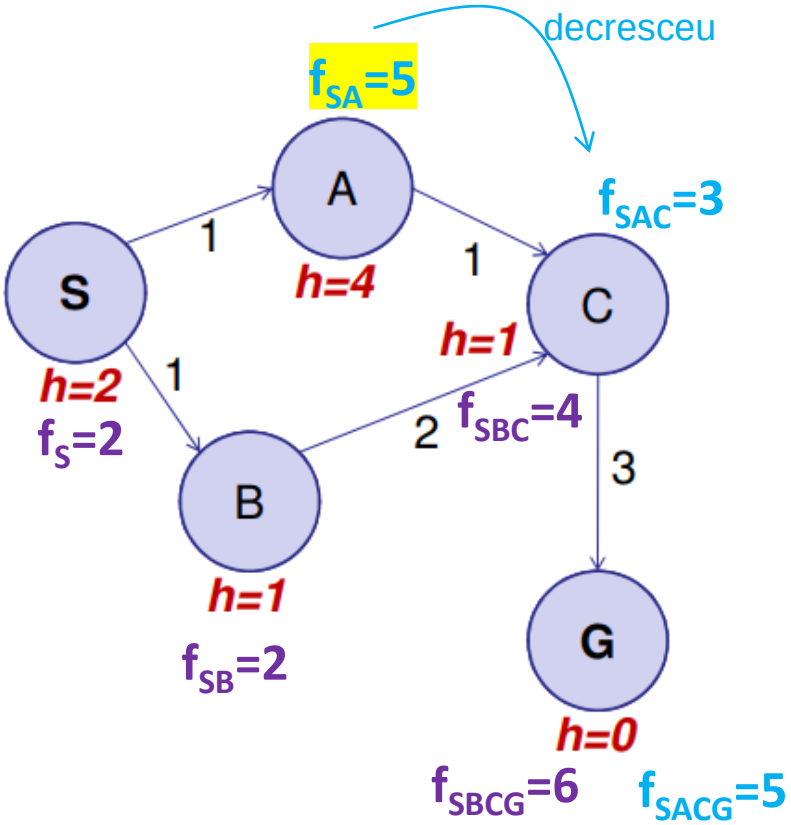
Heurística consistente? NÃO

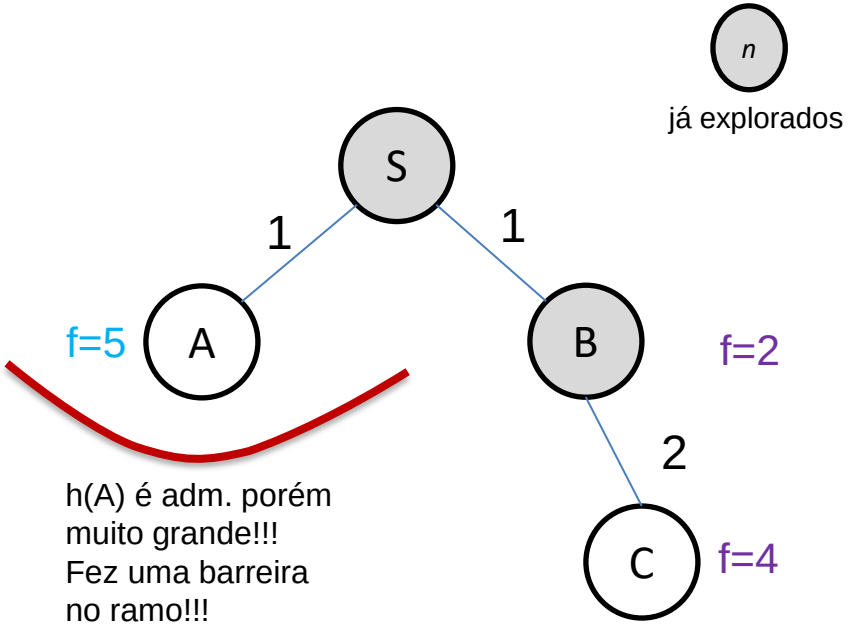
n	n'	h(n)	≤	c(n,n')+h(n')
S	A	2	V	5
S	B	2	V	2
A	C	4	F	2
B	C	1	V	3
C	G	1	V	3

Qual a consequência para a busca?

Heurística Consistente

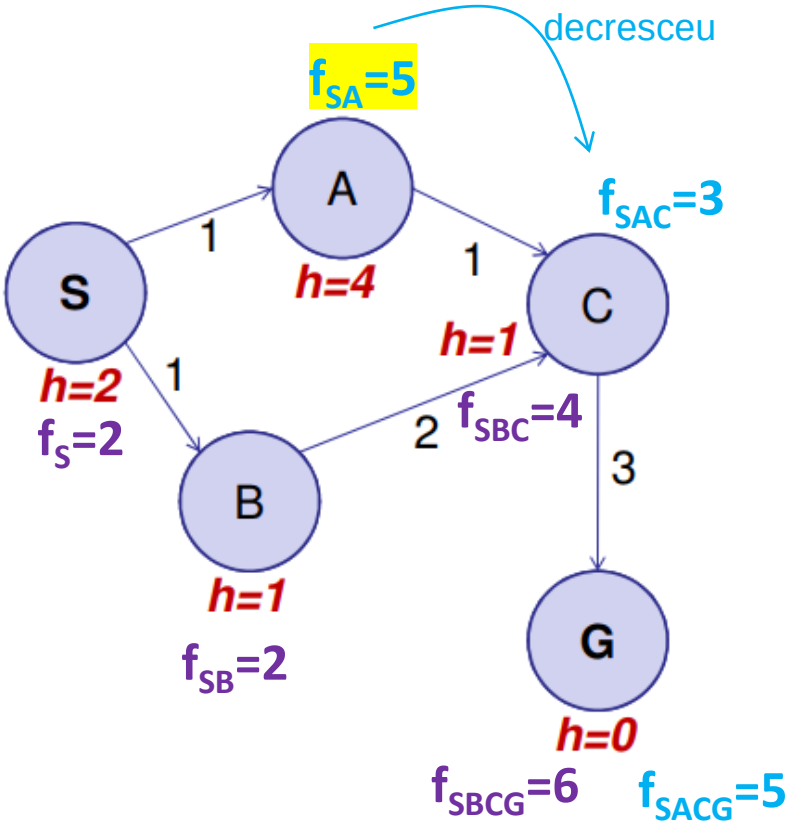
Heurística admissível e inconsistente

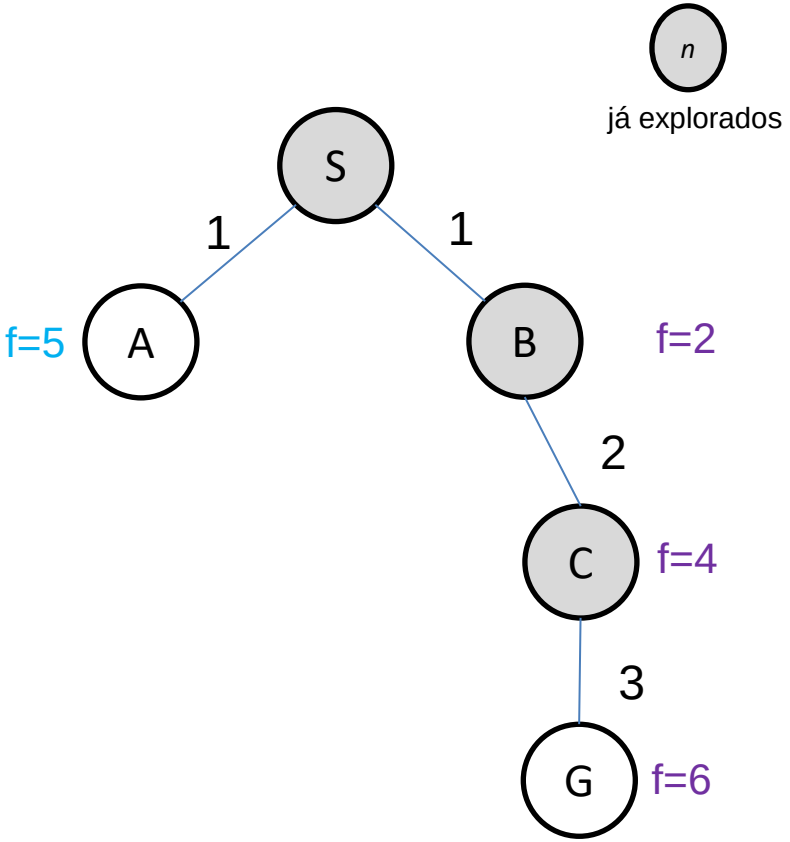




Heurística Consistente

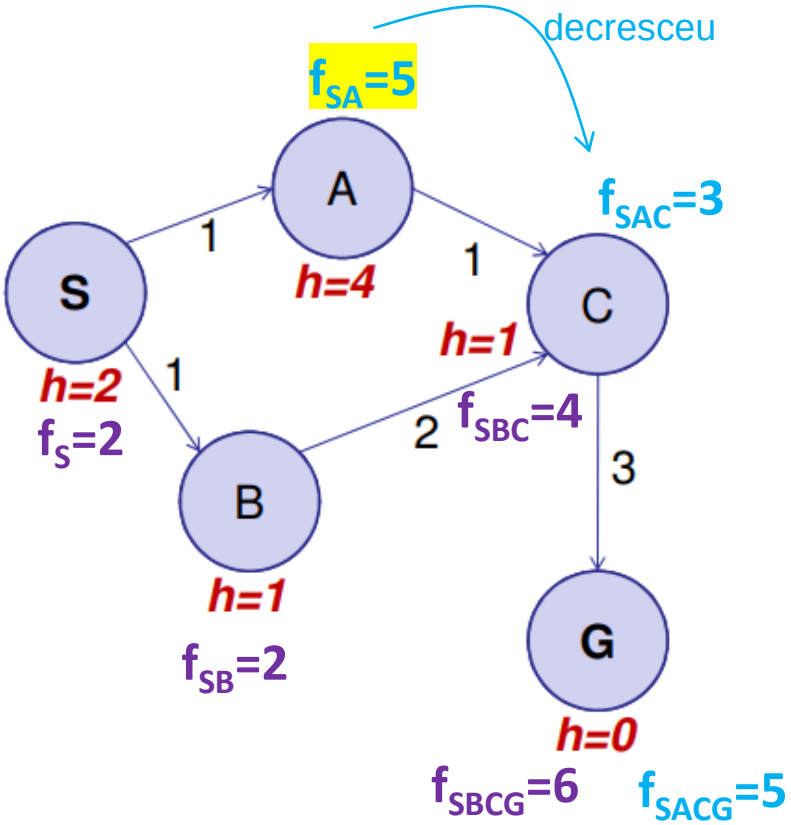
Heurística admissível e inconsistente

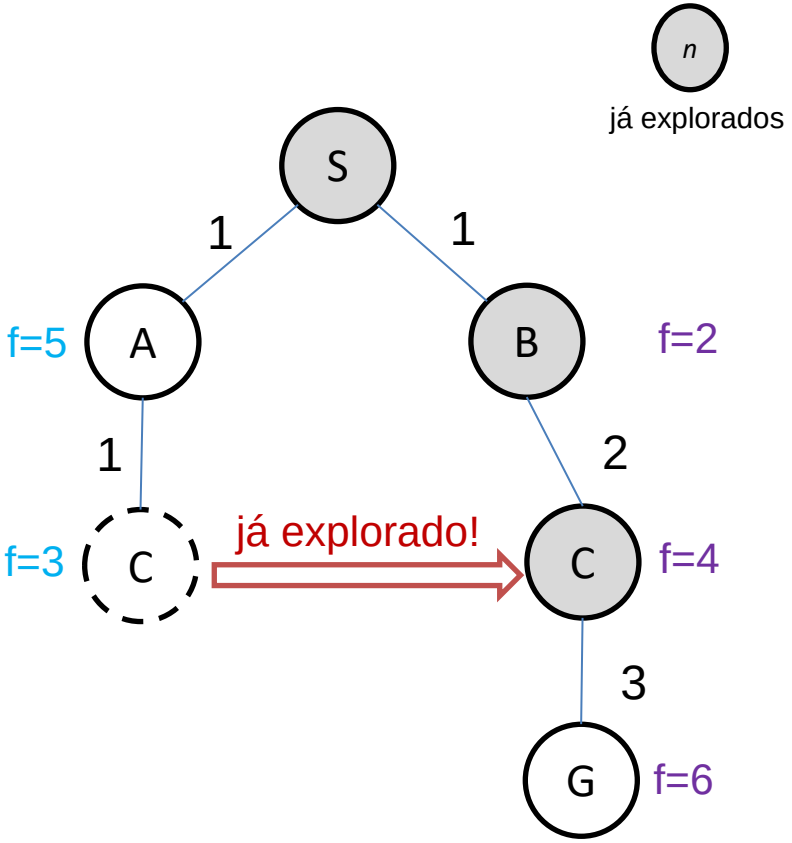




Heurística Consistente

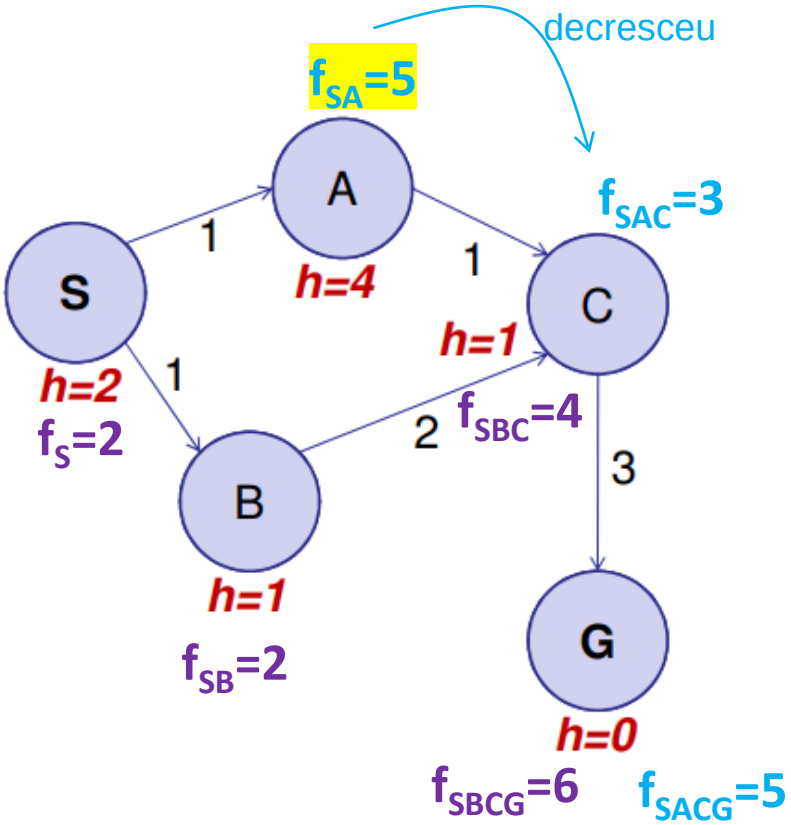
Heurística admissível e inconsistente

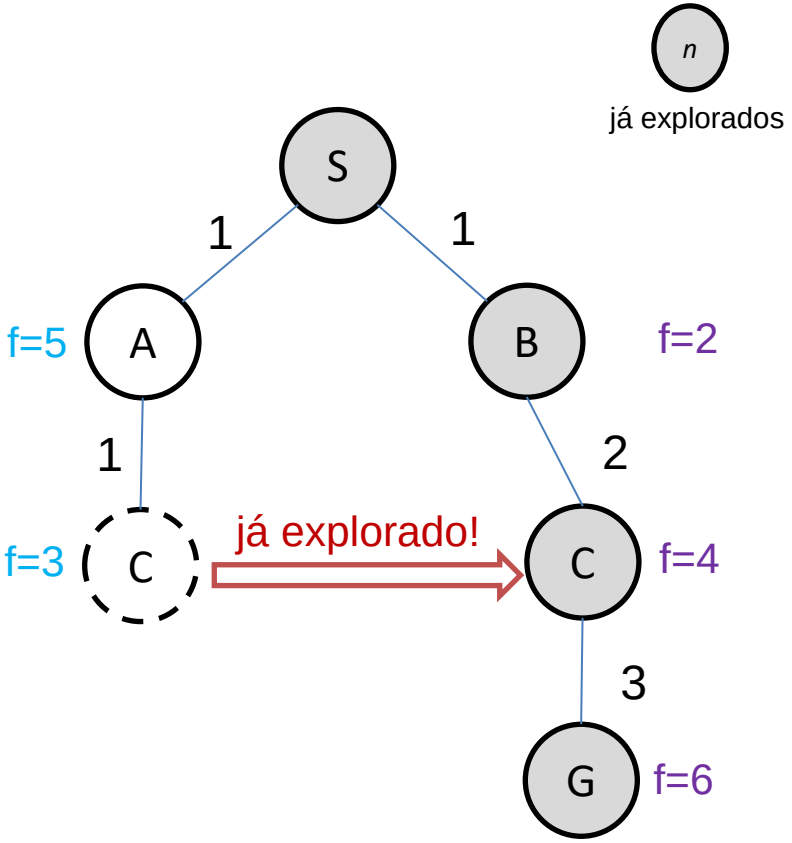




Heurística Consistente

Heurística admissível e inconsistente

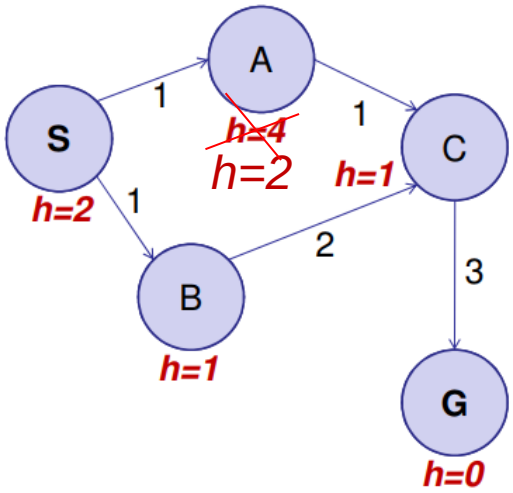




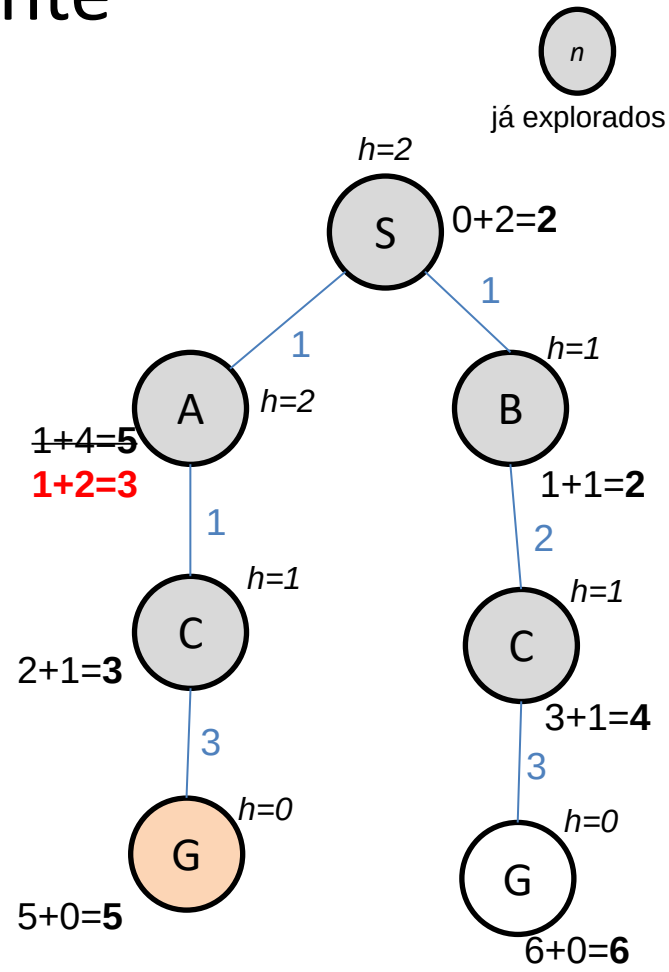
Consequência: encontra um caminho não-ótimo.
Poda caminhos que poderiam ser ótimos.

Heurística Consistente

R: Todo sucessor de n (denominado n') DEVE ter f(n') ≥ f(n)
Diferença entre as heurísticas de n e n' deve ser otimista
h(n) – h(n') ≤ c(n, a, n')



***Corrigindo a heurística**:*
h(A)-h(C) ≤ c(A, a, C)
h(A)-1 ≤ 1
h(A) ≤ 2

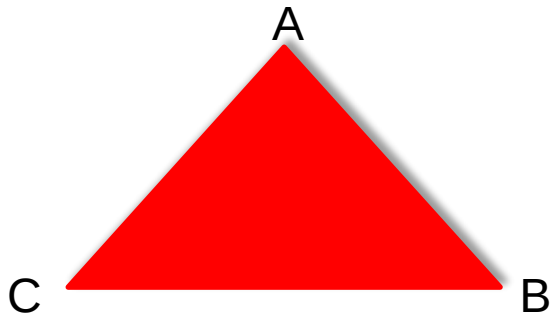


h(S)	− h(A)	≤ c(S, *a*, A)	∴	2 − 2	≤ 1
h(A)	**− h(C)**	**≤ c(A, *a*, C)**	**∴**	**2 − 1**	**≤ 1**
h(C)	− h(G)	≤ c(C, *a*, G)	∴	1 − 0	≤ 3
h(S)	− h(B)	≤ c(S, *a*, B)	∴	2 − 1	≤ 1
h(B)	− h(C)	≤ c(B, *a*, C)	∴	1 − 1	≤ 2
h(C)	− h(G)	≤ c(C, *a*, G)	∴	3 − 0	≤ 3

a é uma ação que leva de n a n'

Heurística Consistente

- Em conclusão: para A* com BUSCA-EM-GRAFO há uma condição adicional para a f(n):
  - ser **CONSISTENTE ou MONOTÔNICA;**
  - i.e. a f(n) deve ser não-decrescente
  - Deve respeitar o teorema da DESIGUALDADE TRIANGULAR



O tamanho de um lado não pode ser maior do que a soma dos tamanhos dos outros dois lados.

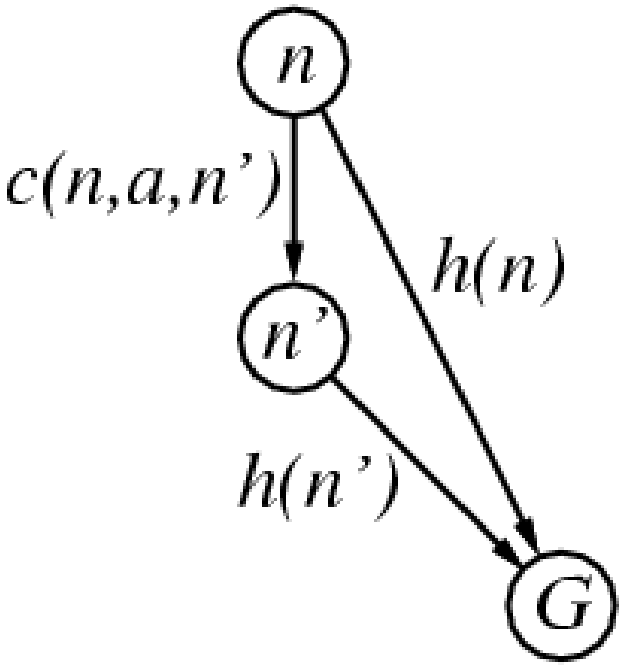
Heurística Consistente (MONOTONICIDADE)

Para *h(n) ser* consistente então **f(n) ≤ f(n')**

$f(n) \leq f(n')$
$g(n) + h(n) \leq g(n') + h(n')$
$g(n) + h(n) \leq g(n) + c(n, a, n') + h(n')$
$h(n) \leq c(n, a, n') + h(n')$

***f(n)* não decresce** ao longo de qualquer caminho, logo:

para todo nó n,
    para cada sucessor n' de n verifica-se
        $h(n) \leq c(n, a, n') + h(n')$

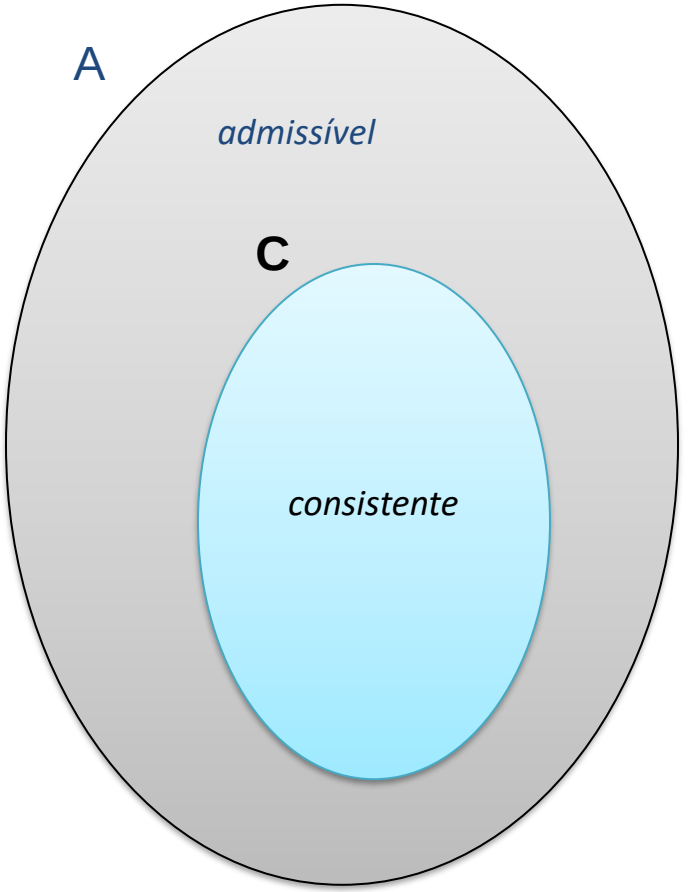


Teorema: Se *h(n)* é consistente, A* é ótima (para busca em grafo)

Heurística consistente

- Consistência implica admissibilidade

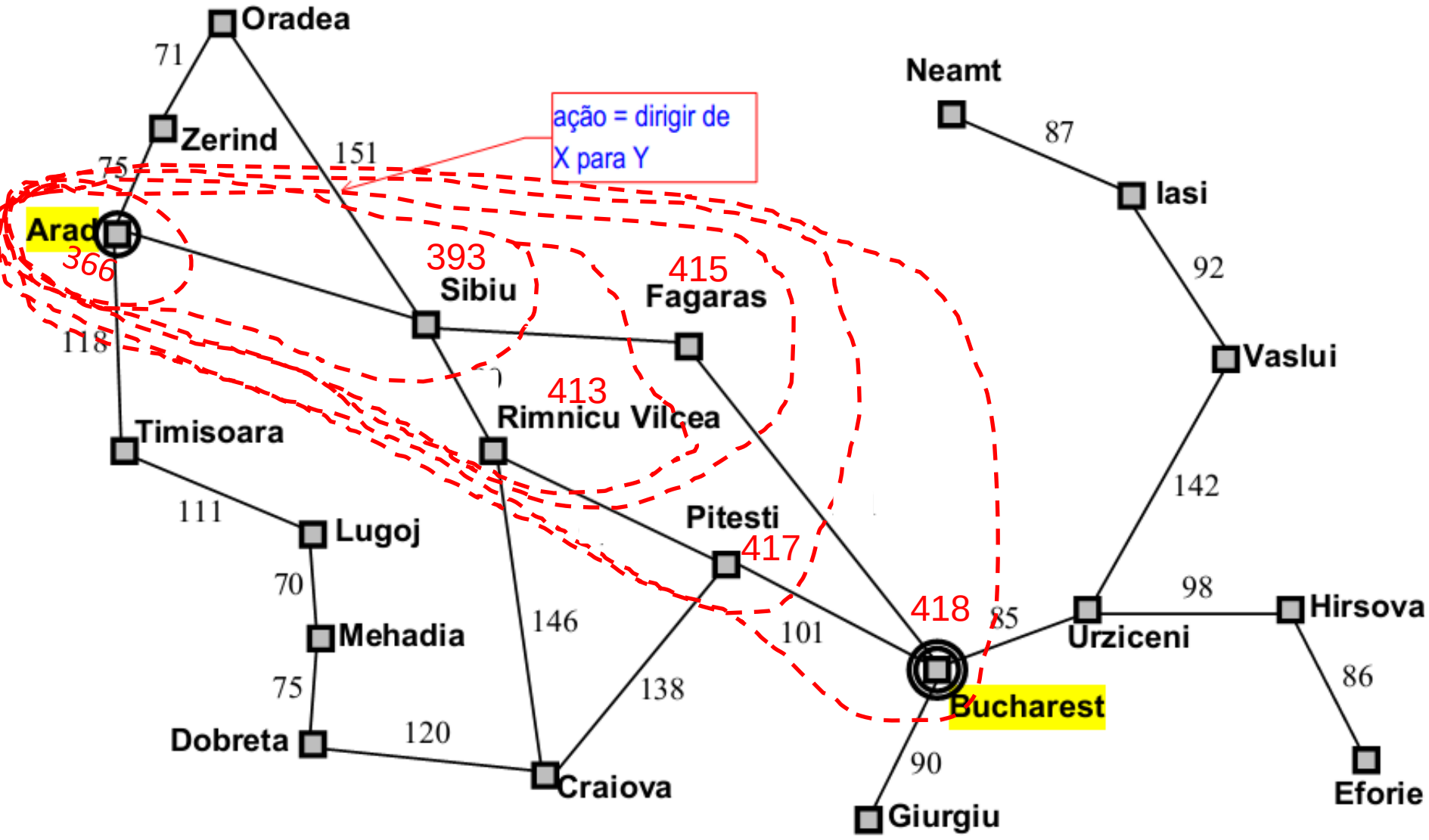
- Porém, admissibilidade não implica em consistência



Propriedades de A*

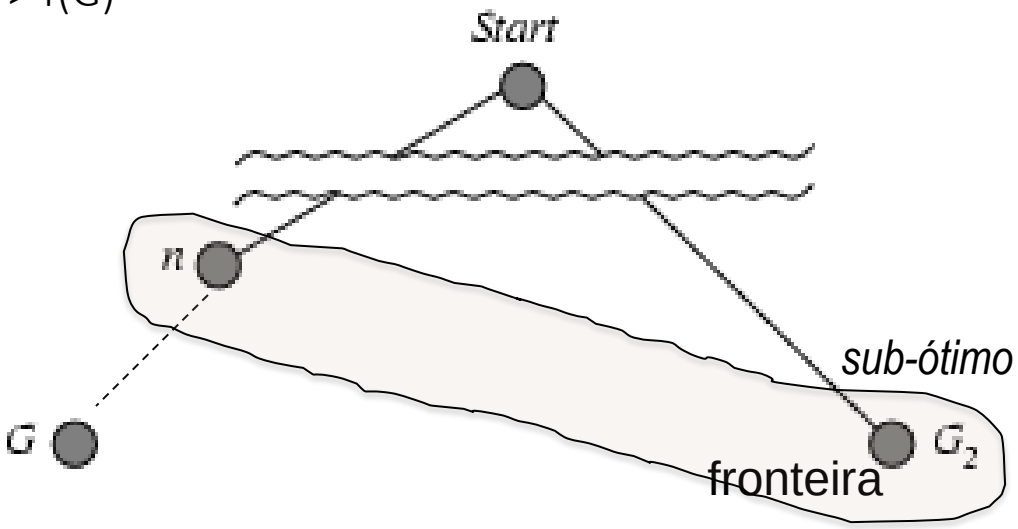
- A* expande nós pela ordem do valor de *f* (nunca decresce)
- Gradualmente vão sendo adicionados os "*f*-contornos" = nós selecionados para expansão: ARAD > SIBIU > ...
- O Contorno *i* possui todos os nós com $f=f_i$, onde $f_i < f_{i+1}$
- *Quando A* seleciona n para expansão significa que já encontrou o caminho de menor custo até ele*
- *A* nunca expande nós que tenham f(n) > C**

A*


Oradea
71
Zerind
75
151
ação = dirigir de
X para Y
Neamt
87
Iasi
92
Vaslui
142
Arad
366
393
Sibiu
415
Fagaras
118
413
Rimnicu Vilcea
Timisoara
111
Lugoj
70
Mehadia
75
Dobreta
120
Craiova
146
138
Pitesti
417
101
418
85
Urziceni
98
Hirsova
86
Eforie
Bucharest
90
Giurgiu

Otimalidade de A* (prova)

Provar que mesmo que haja um objetivo sub-ótimo G_2 na fronteira, A* alcança o objetivo ótimo G, pois $f(G_2) > f(G)$



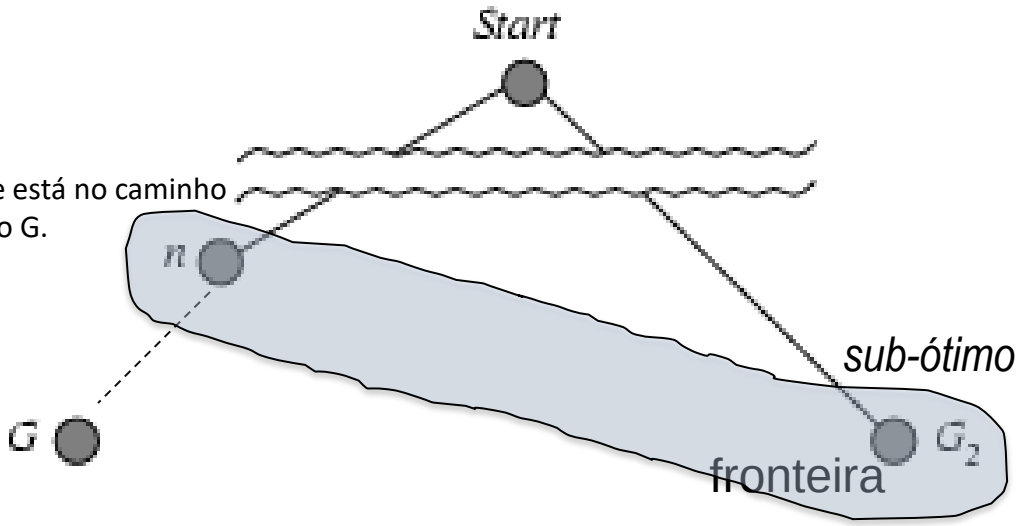
1. $f(G_2) = g(G_2)$ porque $h(G_2) = 0$
2. $f(G) = g(G)$ porque $h(G) = 0$
3. $g(G_2) > g(G)$ porque G_2 é um caminho subótimo
4. de 1 e 2 em 3
   $f(G_2) > f(G)$

logo G será expandido antes

Otimalidade de A* (prova)

Prova: mesmo que um objetivo sub-ótimo (G_2) esteja na fronteira, ele não será explorado nem antes de n nem de G.

n é um nó não expandido na fronteira que está no caminho **mais barato** para chegar ao objetivo ótimo G.



1. $f(G) < f(G_2)$ *do exposto anteriormente*
2. $h(n) \leq h^*(n)$ *porque h é admissível*
3. *incluindo g(n) nos dois lados de (2)*
   $g(n) + h(n) \leq g(n) + h^*(n)$
4. reescreve-se (3)
   $f(n) \leq g(n) + h^*(n)$
5. como $g(n) + h^*(n) = f(G)$, pois g(n) está no caminho ótimo e h*(n) é o custo do caminho ótimo real de n até G, pode-se reescrever (4):
   $f(n) \leq f(G)$
6. e, considerando (1)
   $f(n) \leq f(G) < f(G_2)$

Logo, A* expande os nós nesta ordem $n < G < G_2$

Propriedades de A*

- Completa?
  - Sim, desde que não existam infinitos nós com $f(n) \leq C^*$
    - C^* custo do caminho da solução ótima
    - (A^* expande todos os nós tal que $f(n) < C^*$)
- Tempo?
  - Exponencial: $O(b^{\Delta})$
    - $\Delta = h^* - h$ (*erro absoluto da heurística)*
- Espaço?
  - Guarda todos os nós na memória
    - problema: normalmente explode para espaços de estados grandes
- Ótima?
  - Sim,
    - se heurística for admissível e consistente

Solução de problemas por meio de busca

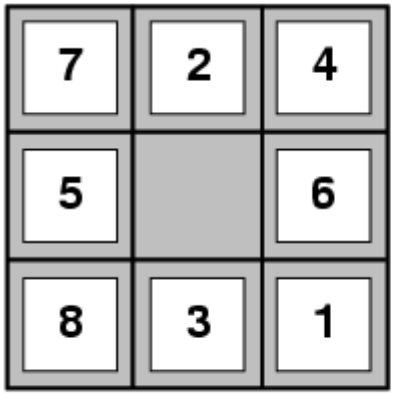
HEURÍSTICAS ADMISSÍVEIS

Heurísticas Admissíveis

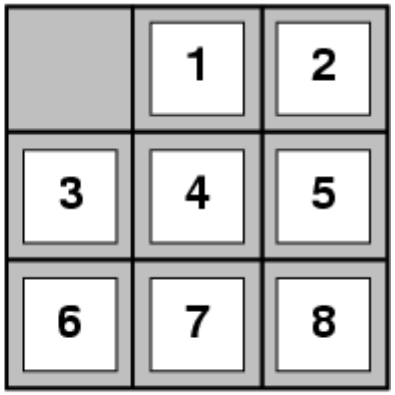
E.g., para o quebra-cabeça de 8 peças

- $h_1(n)$ = número de pedras fora do lugar
- $h_2(n)$ = distância total à la Manhattan

(i.e., número de quadrados da localização desejada de cada pedra)



Start State



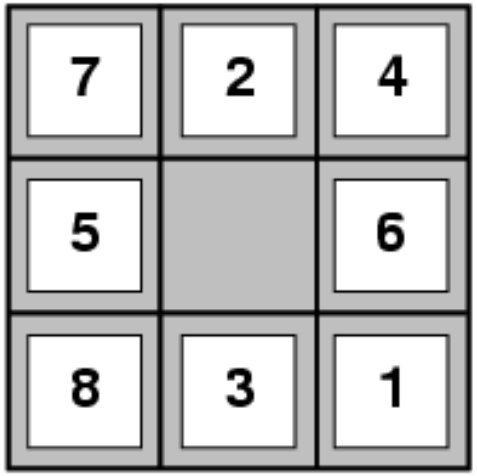
Goal State

- $h_1(S)$ = ?
- $h_2(S)$ = ?

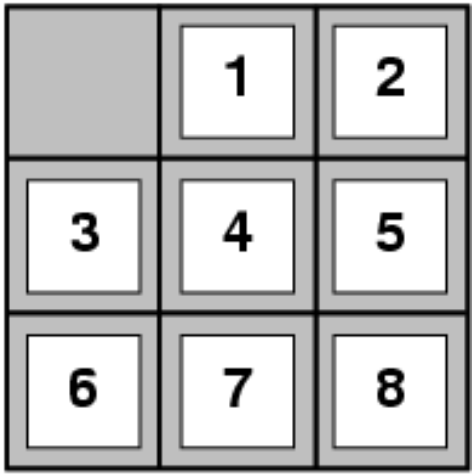
Heurísticas Admissíveis

E.g., para o quebra-cabeça de 8 peças
- $h_1(n)$ = número de pedras fora do lugar
- $h_2(n)$ = distância total à la Manhattan

(i.e., número de quadrados da localização desejada de cada pedra)



Start State



Goal State

- $h_1(S)$ = ? 8
- $h_2(S)$ = ? 3+1+2+2+2+3+3+2 = 18

Dominância

- **Se h2(n) ≥ h1(n) para todo n (ambas admissíveis)**
  - então h2 domina h1
  - h2 é melhor para busca

- Custos de busca típicos (número médio de nós expandidos):

- d=12 BAI = 3.644.035 nós Busca por Aprofundamento Iterativo
  - A*(h1) = 227 nós
  - A*(h2) = 73 nós

- d=24 BAI = muitos nós!!!
  - A*(h1) = 39.135 nós
  - A*(h2) = 1.641 nós

Problemas com menos restrições

- Um problema com menos restrições nas ações é chamado de problema relaxado ➔ relaxação permite obter heurísticas admissíveis/consistentes

- O custo da solução ótima para um **problema relaxado** é uma heurística admissível para o problema original

- h1 = relaxa as regras do quebra-cabeças com 8 peças tal que uma pedra possa ser movimentada para qualquer posição (passando por cima das outras)

- h2 = relaxa as regras para que as pedras possam ir para qualquer quadrado adjacente (menos relaxada que h1)